Comprehensive Overview: `add_cum_oer_metrics_mod_working.py` System

Executive Summary

The `add_cum_oer_metrics_mod_working.py` system computes cumulative Officer Evaluation Report (OER) performance metrics for military officers across their careers. The system processes time-stamped snapshot data (periodic records of officer status) and OER data (evaluations received and given) to calculate various performance indicators that track an officer's evaluation history up to each snapshot date.

Core Concept: For each snapshot date, the system answers: "What was this officer's cumulative performance history as of this date?" TB metrics can be assigned **forward** (most recent `eval_thru_dt`) or **backward** (paint across the rating window via `eval_strt_dt`) based on configuration, with **separate modes** for self vs pool if desired. An optional precompute can store both modes once for faster experimentation.

Key Innovation (2026 Update): All `groupby().apply()` operations have been converted to explicit `@ray.remote` task-based parallelization (501-style approach), enabling full utilization of all 16 CPU cores simultaneously instead of relying on Modin's automatic parallelization. The system now includes real-time progress reporting and automatic warning suppression for cleaner logs.

System Architecture

High-Level Flow

```
Input: df_merged (snapshots + OER data from join_oer_to_snapshots)
    ↓
[Module 0] Data Quality Checks and Cleaning (optional, default: enabled)
    - Detect duplicate OERs (identical data, different eval_id)
    - Detect date overlaps (same officer, overlapping periods)
    - Clean duplicates (keep first eval_id)
    ↓
[Module 1] Compute Cumulative Metrics (always required)
    - Uses @ray.remote for backward lookup parallelization
    ↓
[Optional: Compute DOR from MODE] (if compute_dor_from_mode=True)
    - Computes date of rank (Captain/Major) using most frequent value
    ↓
[Module 2] Add Grade-Based Metrics (optional)
    - Uses @ray.remote for backward lookup parallelization
    ↓
[Module 3] Add Rolling Window Metrics (optional)
    - Uses @ray.remote for officer group processing
    ↓
[Module 4] Add Ratings Given Metrics (optional)
    - Uses @ray.remote for backward lookup parallelization
    ↓
[Module 6] Add Prestige Unit Metrics (optional)
    - Simple cumulative sum (no parallelization needed)
    ↓
[SAVE POINT] Save Intermediate Results (optional)
    - Allows saving progress before pool metrics (long-running step)
    ↓
[Module 5] Add Pool Metrics (optional, parallelized with @ray.remote)
    - Processes snapshot dates in parallel using @ray.remote
    - Includes OER pool metrics
    - Includes prestige pool metrics (if Module 6 enabled)
    ↓
Output: df_with_cum (snapshots with all requested metrics)
```

Key Design Principles

1. **Modularity**: Each module is independent and can be enabled/disabled via boolean flags
2. **Data Quality First**: Automatic detection and cleaning of data quality issues before processing
3. **Temporal Integrity**: All metrics respect snapshot dates—only data available "as of" each date is used
4. **Deduplication**: Each OER is counted exactly once, even if it appears multiple times in the merged data
5. **Vectorization**: Operations are performed on entire arrays/columns, not row-by-row
6. **Explicit Parallelization**: Uses `@ray.remote` for explicit task-based parallelization (501-style), ensuring all cores are utilized

---

Integration with 520 Pipeline

Overview

The `add_cumulative_oer_metrics()` function is called from the **520 Pipeline** notebook in three separate cells, allowing for resumable execution and optimization:

CELL 6A: Basic Metrics Only

Purpose: Compute foundational cumulative metrics (always required)

Configuration:

```
df_with_metrics = add_cumulative_oer_metrics(
    df_merged=df_base,
    include_ratings_given=False,      # Skip (will add in 6B if needed)
    include_grade_metrics=False,      # Skip (will add in 6B if needed)
    include_last4_metrics=False,      # Skip (will add in 6B if needed)
    include_pool_metrics=False,       # Skip (will add in 6C if needed)
    check_data_quality=CELL6_OPTIMIZE.get('check_data_quality', True),
    # ... other parameters ...
)
```

Output Columns (6 columns): - `cum_tb_rcvd_rtr`, `cum_tb_rcvd_snr` - `cum_evals_rcvd_rtr`, `cum_evals_rcvd_snr` - `cum_tb_rcvd_ratio_rtr`, `cum_tb_rcvd_ratio_snr`

Saved As: `df_pipeline_03a_basic_metrics`

Runtime: ~15-30 minutes (with @ray.remote parallelization)

CELL 6B: Optional Metrics

Purpose: Add ratings given, grade-based, and rolling window metrics

Configuration:

```
df_with_optional = add_cumulative_oer_metrics(
    df_merged=df_base,  # Loaded from 6A output
    include_ratings_given=include_ratings,
    include_grade_metrics=include_grade,
    include_last4_metrics=include_last4,
    include_pool_metrics=False,       # Skip (will add in 6C)
    check_data_quality=False,         # Skip (already done in 6A)
    # ... other parameters ...
)
```

Output Columns (additional 20 columns): - Ratings given: `cum_tb_given_rtr`, `cum_tb_given_snr`, `cum_evals_given_rtr`, `cum_evals_given_snr` - Grade-based: `co_grade_tb_rtr`, `co_grade_tb_snr`, `field_grade_tb_rtr`, `field_grade_tb_snr`, etc. (12 columns) - Last4: `last4_tb_rtr`, `last4_tb_snr`, `last4_evals_rtr`, `last4_evals_snr`, etc. (6 columns)

Saved As: `df_pipeline_03b_optional_metrics`

Runtime: ~30-60 minutes (with @ray.remote parallelization)

CELL 6C: Pre-Pool Prep (no pool metrics)

Purpose: Create a reusable pre-pool dataset without pool metrics

Configuration:

```
df_pre_pool = add_cumulative_oer_metrics(
    df_merged=df_base,  # Loaded from 6B output (or 6A if 6B skipped)
    include_ratings_given=False,
    include_grade_metrics=False,
    include_last4_metrics=False,
    include_pool_metrics=False,       # Pre-pool only
    check_data_quality=False,
    # ... other parameters ...
)
```

Saved As: `df_pipeline_03c_pre_pool_metrics`

Runtime: ~minutes to ~1 hour (depends on optional precompute of both TB modes)

CELL 6D: Pool Metrics

Purpose: Add comparative metrics within rating pools (slowest part)

Configuration:

```
df_with_pool = add_cumulative_oer_metrics(
    df_merged=df_base,  # Loaded from 6C output
    include_ratings_given=False,
    include_grade_metrics=False,
    include_last4_metrics=False,
    include_pool_metrics=True,        # ONLY compute pool metrics
    check_data_quality=False,
    # ... other parameters ...
)
```

Output Columns (additional 28+ columns): - Pool means: `pool_cum_tb_mean_rtr`, `pool_cum_eval_mean_rtr`, etc. - Differences: `cum_tb_diff_rtr`, `cum_eval_diff_rtr`, etc. - Ranks: `cum_tb_rank_rtr`, `cum_eval_rank_rtr`, etc. - Normalized ranks: `cum_tb_rank_norm_rtr`, `cum_eval_rank_norm_rtr`, etc. - Plus prestige pool metrics if enabled (14 additional columns)

Saved As: `df_pipeline_03_oer_metrics`

Runtime: ~1-2 hours (with optimized pool metrics)

Resumability

The four-cell design allows for: - **Resumable execution**: If 6D crashes, you can restart with only 6D=True - **Selective computation**: Skip expensive modules if not needed - **Optimization**: Run 6A once, then run 6B multiple times with different settings

---

Parallelization Strategy (501-Style @ray.remote)

Overview

All `groupby().apply()` operations have been converted to explicit `@ray.remote` task-based parallelization, following the pattern used in `501_New_Era.py`. This ensures full utilization of all available CPU cores.

Pattern Used

Before (Modin groupby().apply() - Underutilized Cores):

```
# Modin's automatic parallelization - often only uses 1-2 cores
results = df.groupby(pid_col).apply(process_function)
```

After (@ray.remote - All Cores Utilized):

Ray Parallelization Flow:

```
┌──────────────────────────────────────────────────────────┐
│         RAY PARALLELIZATION PATTERN (501-STYLE)           │
└──────────────────────────────────────────────────────────┘

Input: DataFrame with officer groups

┌──────────────────────────────────────┐
│ Step 1: Extract Officer Groups      │
│ officer_groups = []                  │
│ for officer_id, group in             │
│   df.groupby(pid_col):              │
│     officer_groups.append(           │
│       (officer_id, group)            │
│     )                                │
│                                      │
│ Result: List of (officer_id, group)  │
│         tuples (e.g., 145,000 groups)│
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 2: Store Lookup in Ray Object   │
│         Store                        │
│ lookup_ref = ray.put(lookup_dict)    │
│                                      │
│ (Shared across all workers,          │
│  avoids serialization overhead)     │
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 3: Wrap Function with @ray.     │
│         remote                        │
│ process_remote = ray.remote(         │
│   process_function_standalone        │
│ )                                    │
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 4: Submit ALL Tasks at Once     │
│ (501-style: submit all, get all)    │
│                                      │
│ remote_refs = []                     │
│ for officer_id, group in             │
│   officer_groups:                    │
│     remote_ref = process_remote.     │
│       remote(group, lookup_ref, ...) │
│     remote_refs.append(remote_ref)   │
│                                      │
│ Result: 145,000 tasks submitted      │
│         to Ray cluster               │
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 5: Collect Results with Progress│
│         Reporting                    │
│                                      │
│ pending_refs = list(remote_refs)    │
│ completed_count = 0                  │
│                                      │
│ while pending_refs:                  │
│   ┌──────────────────────────┐      │
│   │ Wait for up to 100 tasks     │   │
│   │ ready_refs, pending_refs =   │   │
│   │   ray.wait(pending_refs,    │   │
│   │     num_returns=100)         │   │
│   └──────────────┬───────────────┘   │
```

```
│                  │                   │
│                  ▼                   │
│   ┌──────────────────────────────┐   │
│   │ Get completed results        │   │
│   │ completed_results = ray.get( │   │
│   │   ready_refs)                │   │
│   │ results_list.extend(         │   │
│   │   completed_results)         │   │
│   └──────────────┬───────────────┘   │
│                  │                   │
│                  ▼                   │
│   ┌──────────────────────────────┐   │
│   │ Report progress (every 1%    │   │
│   │  or 1000 tasks)              │   │
│   │ if completed_count %         │   │
│   │    report_interval == 0:     │   │
│   │   print(f"Completed: {count} │   │
│   │     of {total} ({pct}%)")    │   │
│   └──────────────────────────────┘   │
└──────────────┬──────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 6: Concatenate Results          │
│ results = pd.concat(results_list)    │
│                                      │
│ Result: Single DataFrame with all    │
│         processed groups             │
└──────────────────────────────────────┘

Benefits:
- All 16 cores utilized simultaneously (~98% utilization)
- Efficient memory sharing via Ray object store
- Real-time progress reporting
- Non-blocking result collection
```

Code Implementation:

```python
# Extract officer groups manually
officer_groups = []
for officer_id, group in df.groupby(pid_col, sort=False):
    officer_groups.append((officer_id, group))

# Store lookup dictionaries in Ray object store
lookup_ref = ray.put(lookup_dict)

# Wrap function with @ray.remote
process_remote = ray.remote(process_function_standalone)

# Submit ALL tasks at once (501-style)
remote_refs = []
for officer_id, group in officer_groups:
    remote_ref = process_remote.remote(group, lookup_ref, ...)
    remote_refs.append(remote_ref)

# Get results with progress reporting (non-blocking)
results_list = []
pending_refs = list(remote_refs)
completed_count = 0
report_interval = max(1, min(1000, total_officers // 100))

while pending_refs:
    # Wait for up to 100 tasks to complete
    num_to_wait = min(100, len(pending_refs))
    ready_refs, pending_refs = ray.wait(pending_refs, num_returns=num_to_wait, timeout=None)

    if ready_refs:
        completed_results = ray.get(ready_refs)
        results_list.extend(completed_results)
        completed_count += len(ready_refs)

        # Report progress every 1% or every 1000 tasks
        if completed_count % report_interval == 0 or completed_count == total_officers:
            pct = int(100 * completed_count / total_officers)
            print(f"    📊 Completed: {completed_count:,} of {total_officers:,} ({pct}%)...")

# Concatenate results
results = pd.concat(results_list, ignore_index=False)
```

Benefits

1. **Full Core Utilization**: All 16 cores run simultaneously at ~98% utilization
2. **No Series of DataFrames**: Direct list of DataFrames, faster concatenation
3. **Efficient Memory Sharing**: Lookup dictionaries stored in Ray object store, shared across workers
4. **Automatic Distribution**: Ray handles task distribution across all available CPUs
5. **Progress Reporting**: Real-time progress updates during result collection (reports every 1% or 1000 tasks)
6. **Warning Suppression**: Ray worker warnings about dictionary distribution are suppressed to reduce log clutter

Modules Using @ray.remote

All modules using `@ray.remote` now include real-time progress reporting:

1. **Module 1 (Cumulative Metrics)**: Assignment lookup for cumulative metrics
2. **Module 2 (Grade Metrics)**: Backward lookup for grade-based metrics
3. **Module 3 (Last4 Metrics)**: Rolling window computation per officer
4. **Module 4 (Ratings Given)**: Backward lookup for rater/snr_rater metrics
5. **Module 5 (Pool Metrics)**: Snapshot date processing (already had @ray.remote with progress reporting)

Ray Object Store Usage

Large lookup dictionaries (e.g., `cum_by_officer`, `grade_by_officer`) are stored in Ray's object store using `ray.put()` to avoid serializing them for each task:

```
# Store once in object store
cum_by_officer_ref = ray.put(cum_by_officer)

# Share reference across all tasks (efficient)
for officer_id, group in officer_groups:
    remote_ref = process_remote.remote(group, cum_by_officer_ref, ...)
```

This reduces serialization overhead and memory usage.

Note: Ray may emit informational warnings about distributing dictionary objects. These warnings are automatically suppressed in worker functions and are harmless—they simply indicate that Ray is serializing large objects for distribution to worker processes.

Progress Reporting

All parallel operations now include real-time progress reporting during result collection:

- **Update Frequency**: Reports every 1% completion or every 1000 tasks (whichever is more frequent)
- **Format**: `📊 Completed: X of Y officer groups (Z%) at [timestamp]...`
- **Non-Blocking**: Uses `ray.wait()` to collect results as they complete, allowing progress updates without blocking
- **Batched Collection**: Waits for up to 100 tasks at a time for efficiency

This provides visibility into long-running operations (especially important for 145,000+ officer groups) and helps estimate remaining time.

Module Overview

Module 0: Data Quality Checks

Purpose: Detect and clean data quality issues before processing

Checks: - **Duplicate OERs**: Identical data with different `eval_id` (scanning errors) - **Date Overlaps**: Same officer, overlapping OER date ranges (regulation violations)

Algorithm: - Vectorized overlap detection using NumPy broadcasting ($O(n^2)$ but vectorized) - Deduplication using `drop_duplicates()` with appropriate subset

Time Complexity: $O(n^2)$ per officer for overlap detection, but vectorized

Module 1: Cumulative Metrics (Always Required)

Purpose: Compute foundational cumulative OER metrics **TB assignment mode**: Config-driven
- `TB_ASSIGNMENT_MODE_SELF`: self metrics
- `TB_ASSIGNMENT_MODE_POOL`: pool construction
- `forward`: most recent `eval_thru_dt` ≤ snapshot
- `backward`: apply OER across the rating window via `eval_strt_dt`

Optional precompute: - `TB_PRECOMPUTE_BOTH_MODES=True` stores both modes once (suffixes `_fwd`, `_bwd`)

Output Columns (6 columns): - `cum_tb_rcvd_rtr`, `cum_tb_rcvd_snr`: Cumulative top blocks received - `cum_evals_rcvd_rtr`, `cum_evals_rcvd_snr`: Cumulative evaluations received - `cum_tb_rcvd_ratio_rtr`, `cum_tb_rcvd_ratio_snr`: TB/evals ratios

Algorithmic Flow:

```
┌──────────────────────────────────────────────────────────────┐
│         MODULE 1: CUMULATIVE METRICS COMPUTATION           │
└──────────────────────────────────────────────────────────────┘

Input: df_merged (snapshots + OER data)

┌──────────────────────────────────────┐
│ Step 1: Extract & Deduplicate OERs   │
│ - Extract OER rows from df_merged    │
│ - Remove duplicates (same eval_id)   │
│ - Sort by (officer_id, effective_date)│
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 2: Create Contribution Flags    │
│ For each OER:                        │
│   - rater_box == 50.0? → TB_rtr = 1  │
│   - snr_rater_box == 50.0? → TB_snr=1│
│   - Any numeric value? → Eval = 1     │
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 3: Compute Cumulative Sums      │
│ For each officer:                    │
│   ┌──────────────────────────────┐   │
│   │ Sort OERs by thru_date       │   │
│   │                              │   │
│   │ OER1: TB=1 → cum_tb = 1     │   │
│   │ OER2: TB=0 → cum_tb = 1     │   │
│   │ OER3: TB=1 → cum_tb = 2     │   │
│   │ OER4: TB=1 → cum_tb = 3     │   │
│   │                              │   │
│   │ Store: cum_by_date[date]     │   │
│   └──────────────┬───────────────┘   │
└──────────────────┼───────────────────┘
                   │
                   ▼
┌──────────────────────────────────────┐
│ Step 4: Assignment Lookup            │
│ (Parallelized with @ray.remote)      │
│                                      │
│ For each officer group:              │
│   ┌──────────────────────────────┐   │
│   │ For each snapshot:           │   │
│   │   ┌────────────────────────┐ │   │
│   │   │ Binary search:         │ │   │
│   │   │ Find last OER where    │ │   │
│   │   │   effective_date ≤     │ │   │
│   │   │   snapshot_date        │ │   │
│   │   └────────────┬───────────┘ │   │
│   │                │             │   │
│   │                ▼             │   │
│   │   ┌────────────────────────┐ │   │
│   │   │ If found:              │ │   │
│   │   │   cum_tb = cum_by_     │ │   │
│   │   │            date[index] │ │   │
│   │   │ Else:                  │ │   │
│   │   │   cum_tb = 0           │ │   │
│   │   └────────────────────────┘ │   │
│   └──────────────────────────────┘   │
└──────────────┬───────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 5: Compute Ratios               │
│ For each snapshot:                   │
│   cum_tb_ratio = cum_tb / cum_evals  │
```

```
|   (Handle division by zero)           |
└──────────────────────────────────────┘
                 |
                 |
                 ▼
         Output: df_with_metrics
         (6 new columns added)
```

Algorithm: 1. Extract and deduplicate OERs 2. Create contribution flags (binary: does this OER contribute?) 3. Compute cumulative sums per officer 4. **Assignment lookup** (parallelized with @ray.remote): Attach cumulative metrics using the configured TB assignment mode

Time Complexity: - Cumulative sum: O(n) where n = number of OERs - Assignment lookup: O(m log k) where m = snapshots, k = OERs per officer (parallelized across officers)

Parallelization: Uses @ray.remote for officer group processing

Module 2: Grade-Based Metrics

Purpose: Compute metrics for specific rank periods (Captain, Major)

Output Columns (12 columns): - CO grade (Captain): `co_grade_tb_rtr`, `co_grade_tb_snr`, `co_grade_evals_rtr`, `co_grade_evals_snr` - Field grade (Major): `field_grade_tb_rtr`, `field_grade_tb_snr`, `field_grade_evals_rtr`, `field_grade_evals_snr` - Ratios: `co_grade_tb_ratio_rtr`, `co_grade_tb_ratio_snr`, `field_grade_tb_ratio_rtr`, `field_grade_tb_ratio_snr`

Algorithm: 1. Filter OERs by date of rank (DOR) to determine rank at time of OER 2. Compute cumulative sums separately for CO grade and field grade periods 3. **Backward lookup** (parallelized with @ray.remote): Find grade metrics as of each snapshot date

Time Complexity: Same as Module 1, but with additional filtering step

Parallelization: Uses @ray.remote for officer group processing

Module 3: Rolling Window (Last N) Metrics

Purpose: Compute metrics for the last N evaluations (rolling window)

Output Columns (6 columns): - `last4_tb_rtr`, `last4_tb_snr`: Top blocks in last 4 evaluations - `last4_evals_rtr`, `last4_evals_snr`: Evaluations in last 4 (max 4) - `last4_tb_ratio_rtr`, `last4_tb_ratio_snr`: TB/evals ratios

Algorithm: 1. Sort OERs by officer and completion date 2. For each snapshot, find OERs with `thru_date <= snapshot_date` 3. Take last N OERs (window size, default 4) 4. Sum TBs and count evals in window

Time Complexity: O(m k) where m = snapshots, k = OERs per officer (parallelized across officers)

Parallelization: Uses @ray.remote for officer group processing

Module 4: Ratings Given Metrics

Purpose: Compute metrics for evaluations given as rater/senior rater

Output Columns (4 columns): - `cum_tb_given_rtr`, `cum_tb_given_snr`: Cumulative top blocks given - `cum_evals_given_rtr`, `cum_evals_given_snr`: Cumulative evaluations given

Algorithm: 1. Extract OERs where officer was the rater/snr_rater (instead of rated officer) 2. Compute cumulative sums grouped by rater/snr_rater ID 3. **Backward lookup** (parallelized with @ray.remote): Find metrics as of each snapshot date

Time Complexity: Same as Module 1, but grouped by rater instead of rated officer

Parallelization: Uses @ray.remote for officer group processing

Module 5: Pool Metrics (Comparative Metrics)

Purpose: Compute comparative metrics within rating pools (officers with same rater/snr_rater)

Output Columns (28+ columns): - Pool means: `pool_cum_tb_mean_rtr`, `pool_cum_eval_mean_rtr`, etc. - Differences: `cum_tb_diff_rtr`, `cum_eval_diff_rtr`, etc. - Ranks: `cum_tb_rank_rtr`, `cum_eval_rank_rtr`, etc. - Normalized ranks: `cum_tb_rank_norm_rtr`, `cum_eval_rank_norm_rtr`, etc. - **Planned additions**: - Percentile ranks (ordering): `*_rank_pct_*` - Within-pool z-scores (magnitude): `*_zpool_*` - Plus prestige pool metrics if enabled (14 additional columns)

Algorithmic Flow:

```
┌──────────────────────────────────────────────────────────────┐
|         MODULE 5: POOL METRICS COMPUTATION                   |
└──────────────────────────────────────────────────────────────┘

Input: df_with_metrics (snapshots with cumulative metrics)
       OER data with rater/snr_rater IDs

┌──────────────────────────────────────┐
| Step 1: Create Rater Lookup          |
| For each officer, for each date:     |
|   Find most recent start_date ≤ snap |
|   Keep only if snap ≤ thru_date      |
|                                      |
|   Result: lookup[officer][date] =    |
|           (rater_id, snr_rater_id)   |
└──────────────┬───────────────────────┘
               |
               ▼
┌──────────────────────────────────────┐
| Step 2: Get Unique Snapshot Dates    |
| dates = unique(snapshot_dates)       |
| (e.g., 200+ dates)                   |
└──────────────┬───────────────────────┘
               |
               ▼
┌──────────────────────────────────────┐
| Step 3: Parallelize Across Dates     |
| (Submit all tasks to Ray)            |
|                                       |
| For each snapshot_date:               |
|   ┌──────────────────────────┐       |
|   | @ray.remote task:        |       |
|   |                          |       |
|   | a. Filter snapshots:     |       |
|   |    Get all rows where    |       |
|   |    snapshot_date == date |       |
|   |                          |       |
|   | b. Group by rater/snr_rater:|    |
|   |    Create pools:         |       |
|   |    pool[rater_id] = [    |       |
|   |      officers with same  |       |
|   |      rater on this date  |       |
|   |    ]                      |       |
|   |                          |       |
|   | c. For each pool:        |       |
|   |    ┌────────────────────┐ |       |
|   |    | Compute pool mean: | |       |
|   |    | mean = mean(       | |       |
|   |    |   cum_tb for all   | |       |
|   |    |   officers in pool | |       |
|   |    |   excluding self   | |       |
|   |    | )                  | |       |
|   |    └────────┬───────────┘ |       |
|   |             |              |       |
|   |             ▼              |       |
|   |    ┌────────────────────┐ |       |
|   |    | Compute difference: | |      |
|   |    | diff = officer_tb - | |      |
|   |    |        pool_mean   | |       |
|   |    └────────┬───────────┘ |       |
|   |             |              |       |
|   |             ▼              |       |
|   |    ┌────────────────────┐ |       |
|   |    | Compute rank:      | |       |
|   |    | rank = position of | |       |
|   |    |        officer_tb in | |     |
|   |    |        sorted pool | |       |
|   |    |        (1 = highest) | |     |
|   |    └────────┬───────────┘ |       |
```

```
│   │               │              │   │
│   │               ▼              │   │
│   │    ┌──────────────────────┐  │   │
│   │    │ Normalized rank:     │  │   │
│   │    │ norm_rank = rank /   │  │   │
│   │    │            pool_size│  │   │
│   │    │            (0-1)     │  │   │
│   │    └──────────────────────┘  │   │
│   │                              │   │
│   │ d. Return pool metrics       │   │
│   │    for this date             │   │
│   └──────────────────────────────┘   │
└──────────────┬──────────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 4: Collect Results              │
│ (Get all tasks from Ray)             │
│ Combine results from all dates       │
└──────────────┬──────────────────────────┘
               │
               ▼
┌──────────────────────────────────────┐
│ Step 5: Merge Back to Main DataFrame │
│ Merge pool metrics back to           │
│ df_with_metrics by (officer, date)   │
└──────────────┬──────────────────────────┘
               │
               ▼
        Output: df_with_pool_metrics
        (28+ new columns added)
```

Algorithm: 1. Create overlap-based assignments: most recent start_date ≤ snapshot, keep only if snapshot ≤ thru_date 2. **For each snapshot date** (parallelized with @ray.remote): - Determine pool membership (officers with same rater/snr_rater) - Compute pool means (excluding self if `exclude_self=True`) - Compute differences, ranks, normalized ranks - **Planned**: percentile ranks (order only) + within-pool z-scores (magnitude) 3. Merge results back into main DataFrame

Time Complexity: - Lookup creation: O(n) where n = OER rows - Per snapshot date: $O(p^2)$ where p = pool size (vectorized) - Overall: $O(d\ p^2)$ where d = snapshot dates, p = average pool size (parallelized across dates)

Parallelization: Uses @ray.remote for snapshot date processing (submit all, get all)

Optimized Version (`_add_pool_metrics_optimized`):

As of January 2025, an optimized version of pool metrics computation is available that addresses performance and memory issues:

Key Improvements: 1. **Precomputed Rater Assignments**: Uses `pd.merge_asof()` on `eval_strt_dt` then filters to `snapshot_date <= eval_thru_dt` to enforce overlap-based pools, all in a single vectorized pass. 2. **Vectorized Pool Computations**: Uses pandas `groupby().transform()` and vectorized operations instead of iterative per-pool computations. 3. **Memory Efficiency**: Avoids Ray worker OOM (Out-Of-Memory) issues by eliminating the need for large lookup dictionaries in Ray object store. 4. **Performance**: Expected 10-100x speedup compared to the original implementation (from ~33 hours to minutes/hours).

Algorithmic Flow (Optimized):

```
┌──────────────────────────────────────────────────────────────┐
|    MODULE 5: POOL METRICS COMPUTATION (OPTIMIZED)          |
└──────────────────────────────────────────────────────────────┘

Input: df_with_metrics (snapshots with cumulative metrics)
       OER data with rater/snr_rater IDs

┌──────────────────────────────────────┐
| Step 1: Precompute Rater Assignments |
| (VECTORIZED - Single Operation)      |
|                                      |
| Use pd.merge_asof() on start_date,   |
| then filter to snap ≤ thru_date      |
|                                      |
|   merge_asof(                        |
|     snapshot_officer_pairs,          |
|     oer_rows,                        |
|     left_on=snapshot_date,           |
|     right_on=eval_strt_dt,           |
|     by=officer,                      |
|     direction='backward'             |
|   )                                  |
|   then filter to snap ≤ eval_thru_dt |
|                                      |
|   Result: All rater assignments      |
|           computed in one pass       |
└──────────────┬───────────────────────┘
               |
               ▼
┌──────────────────────────────────────┐
| Step 2: Join Assignments to Main DF  |
| Merge precomputed assignments back   |
| to main DataFrame                    |
└──────────────┬───────────────────────┘
               |
               ▼
┌──────────────────────────────────────┐
| Step 3: Vectorized Pool Computations |
| (No Ray Parallelization Needed)      |
|                                      |
| For rater pools:                     |
|   ┌──────────────────────────────┐   |
|   | Group by (date, rater_id):  |   |
|   |   - Pool sizes: size()      |   |
|   |   - Pool means: transform() |   |
|   |   - Differences: vectorized |   |
|   |   - Ranks: scipy.rankdata() |   |
|   |   - Norm ranks: vectorized  |   |
|   └──────────────────────────────┘   |
|                                      |
| For senior rater pools:              |
|   ┌──────────────────────────────┐   |
|   | Group by (date, snr_rater): |   |
|   |   (same operations)         |   |
|   └──────────────────────────────┘   |
|                                      |
| For prestige pools (if enabled):     |
|   ┌──────────────────────────────┐   |
|   | (same vectorized approach)  |   |
|   └──────────────────────────────┘   |
└──────────────┬───────────────────────┘
               |
               ▼
        Output: df_with_pool_metrics
        (28+ new columns added)
```

Time Complexity (Optimized): - Rater assignment precomputation: O(n log n) where n = snapshot rows (merge_asof sort + merge) - Pool computations: O(n) where n = snapshot rows (vectorized groupby operations) - Overall: O(n log n) - much better than original O(d p^2) where d = dates, p = pool size

Configuration: - Controlled by `use_optimized_pool_metrics` parameter (default: `True`) - Set in `CELL6C_PARAMS` or `CELL6_OPTIMIZE` in `pipeline_config.py` - Original version (`_add_pool_metrics`) is preserved for comparison

Module 6: Prestige Unit Metrics

Purpose: Compute cumulative metrics for quarters spent in prestige units

Output Columns (4 columns): - `cum_prestige_quarters`: Cumulative count of prestige quarters - `cum_total_quarters`: Cumulative count of total quarters - `mean_prestige_quarters`: Cumulative mean (prestige/total) - `prestige_quarters_ratio`: Same as mean (for consistency)

Algorithm: 1. Sort by officer and snapshot date 2. Compute cumulative sum of `prestige_unit` flag (0 or 1) 3. Compute cumulative count of snapshots 4. Compute ratio (prestige/total)

Time Complexity: O(n) where n = snapshot rows (simple cumulative sum)

Parallelization: None needed (fast operation)

Algorithmic Complexity

Time Complexity Summary

Module	Operation	Time Complexity	Parallelization
Module 0	Overlap detection	$O(n^2)$ per officer (vectorized)	None
Module 1	Cumulative sum	O(n)	None
Module 1	Assignment lookup	O(m log k)	@ray.remote (officers)
Module 2	Grade filtering	O(n)	None
Module 2	Backward lookup	O(m log k)	@ray.remote (officers)
Module 3	Rolling window	O(m k)	@ray.remote (officers)
Module 4	Backward lookup	O(m log k)	@ray.remote (officers)
Module 5 (Original)	Lookup creation	O(n)	None
Module 5 (Original)	Pool computation	$O(d\ p^2)$	@ray.remote (dates)
Module 5 (Optimized)	Rater assignment	O(n log n)	None (vectorized)
Module 5 (Optimized)	Pool computation	O(n)	None (vectorized)
Module 6	Cumulative sum	O(n)	None

Where: - `n` = total number of OER rows - `m` = total number of snapshot rows - `k` = average number of OERs per officer - `d` = number of unique snapshot dates - `p` = average pool size (officers per rater/snr_rater)

Space Complexity

- **Lookup dictionaries**: O(o k) where o = number of officers, k = OERs per officer
- **Ray object store**: Stores lookup dictionaries once, shared across workers
- **Intermediate DataFrames**: O(m) where m = snapshot rows
- **Overall**: O(o k + m) - dominated by lookup dictionaries and snapshot data

Performance Characteristics

Runtime Estimates (with @ray.remote parallelization)

Module	Typical Runtime	Notes
Module 0 (Data Quality)	1-2 hours	Can be skipped if data already cleaned
Module 1 (Cumulative)	15-30 minutes	Always required
Module 2 (Grade)	10-15 minutes	Optional
Module 3 (Last4)	10-15 minutes	Optional
Module 4 (Ratings Given)	15-20 minutes	Optional
Module 5 (Pool Metrics - Original)	1-2 hours (or 30+ hours if OOM)	Optional, slowest part, memory issues
Module 5 (Pool Metrics - Optimized)	Minutes to ~1 hour	Optional, 10-100x faster, no memory issues
Module 6 (Prestige)	< 1 minute	Optional, very fast

Total Runtime (all modules enabled): - **CELL 6A**: ~15-30 minutes (basic metrics only) - **CELL 6B**: ~30-60 minutes (optional metrics) - **CELL 6C (Pre-pool)**: Minutes to ~1 hour (depends on optional TB mode precompute) - **CELL 6D (Optimized)**: Minutes to ~1 hour (default as of January 2025)

Previous Runtime (before @ray.remote): 4+ hours total

Speedup: - ~2-4x faster with @ray.remote parallelization (original pool metrics) - ~10-100x faster with optimized pool metrics (vectorized operations)

Memory Usage

- **Peak Memory**: ~40-60 GB (with 40 GB Ray object store)
- **Lookup Dictionaries**: ~10-20 GB (stored in Ray object store)
- **Snapshot Data**: ~5-10 GB
- **Intermediate Results**: ~10-20 GB

Bottlenecks

1. **Module 5 (Pool Metrics - Original)**: Slowest part due to $O(d\ p^2)$ complexity and Ray OOM issues
    - **Solution**: Use optimized version (`use_optimized_pool_metrics=True`) for 10-100x speedup
2. **Lookup Dictionary Creation**: O(n) but can be slow for large datasets
    - **Solution**: Optimized version uses vectorized `merge_asof()` instead

3. **Data Quality Checks**: O(n²) per officer for overlap detection (can be skipped)

Data Flow and Processing

Input Data Structure

The system expects a merged DataFrame (`df_merged`) from `join_oer_to_snapshots_working.py` containing:

Snapshot Columns: - `pid_pde`: Officer identifier - `snpsht_dt`: Snapshot date (the "as of" date for metrics) - Other officer attributes (rank, unit, etc.)

OER Columns: - `rated_ofcr`: Officer who was evaluated (should match `pid_pde`) - `eval_thru_dt`: OER completion date (end of rating period) - `eval_strt_dt`: OER start date - `eval_id`: Unique OER identifier - `rater_box`: Rater's box score (50.0 = top block) - `snr_rater_box`: Senior rater's box score (50.0 = top block) - `rater`: PID of the rater (for "ratings given" metrics) - `snr_rater`: PID of the senior rater (for "ratings given" metrics)

Optional Columns: - `prestige_unit`: Binary indicator (1 = prestige unit, 0 = not) - `dor_cpt`, `dor_maj`: Date of rank (Captain, Major) - `rank_pde`: Rank (for DOR computation) - `ppln_pgrd_eff_dt`: Promotion effective date (for DOR computation)

Processing Flow

1. **Data Quality Checks** (Module 0):
   - Extract OER rows
   - Detect duplicates and overlaps
   - Clean duplicates if requested
2. **Cumulative Metrics** (Module 1):
   - Extract unique OERs
   - Create contribution flags
   - Compute cumulative sums per officer
   - Backward lookup to snapshots (parallelized)
3. **Optional Modules** (2-4, 6):
   - Similar pattern: extract, compute, backward lookup
   - Each module adds columns to the DataFrame
4. **Pool Metrics** (Module 5):
   - Create rater/snr_rater lookup dictionaries
   - Process each snapshot date in parallel
   - Compute pool statistics
   - Merge results back

Output Data Structure

The output DataFrame (`df_with_cum`) contains: - All original columns from `df_merged` - Cumulative metric columns (Module 1) - Optional metric columns (Modules 2-4, 6) - Pool metric columns (Module 5)

Total Columns: ~50-70 columns (depending on enabled modules)

Configuration and Optimization

CELL6_OPTIMIZE Dictionary

The 520 pipeline uses a `CELL6_OPTIMIZE` dictionary to control which modules run:

```
CELL6_OPTIMIZE = {
    'check_data_quality': False,      # Skip if data already cleaned
    'report_duplicates': False,      # Skip duplicate reporting
    'report_overlaps': False,         # Skip overlap reporting
    'clean_duplicates': True,         # Keep True to remove duplicates
    'include_ratings_given': True,     # Set False if not needed
    'include_grade_metrics': True,     # Set False if not needed
    'include_last4_metrics': True,    # Set False if not needed
    'include_pool_metrics': True,     # Set False if not needed (saves 1-2 hours)
}
```

TB Assignment Mode + NaT Policy

These are defined in `pipeline_config.py` and passed into `add_cumulative_oer_metrics()`:

- `TB_ASSIGNMENT_MODE_SELF`:
  - `'forward'`: most recent `eval_thru_dt` ≤ snapshot
  - `'backward'`: paint TB outcomes across the rating window via `eval_strt_dt`
- `TB_ASSIGNMENT_MODE_POOL`:
  - `'forward'`: most recent `eval_thru_dt` ≤ snapshot
  - `'backward'`: paint TB outcomes across the rating window via `eval_strt_dt`
- `TB_PRECOMPUTE_BOTH_MODES`:
  - If `True`, compute both modes once and store suffixed columns (`*_fwd`, `*_bwd`)
- `OER_NAT_POLICY`: how to handle missing `eval_strt_dt` / `eval_thru_dt`
  - `'drop_low_impute_high'` (default), `'drop'`, or `'impute'`
- `OER_NAT_DROP_THRESHOLD`: threshold for switching from drop → impute (default 1%)

Optimization Tips

1. **Skip Data Quality Checks**: If data is already cleaned, set `check_data_quality=False` (saves 1-2 hours)
2. **Skip Unused Modules**: Disable modules you don't need:
   - No grade analysis? Set `include_grade_metrics=False`
   - No pool analysis? Set `include_pool_metrics=False` (saves 1-2 hours)
3. **Resumable Execution**: Use CELL 6A, 6B, 6C design to resume after crashes
4. **Save Intermediate Results**: Use `save_intermediate_filename` parameter to save before pool metrics

Ray Configuration

The system automatically initializes Ray with: - **CPUs**: 16 (detected automatically) - **Object Store**: 40 GB (for large lookup dictionaries) - **Temp Directory**: `/tmp/ray_tmp`

Ray cleanup is performed: - After Module 1 (large computation) - After Module 5 (long-running step) - At end of function

User Experience Enhancements

Progress Reporting: - All parallel operations display real-time progress during result collection - Format: `📊 Completed: X of Y officer groups (Z%) at [timestamp]...` - Updates appear every 1% completion or every 1000 tasks (whichever is more frequent) - Helps estimate remaining time for long-running operations (especially important for 145,000+ officer groups)

Warning Suppression: - Ray's informational warnings about dictionary distribution are automatically suppressed in worker functions - These warnings are harmless but can generate 100,000+ messages, cluttering logs - Suppression is implemented at the worker function level to ensure clean output

---

Cross-References

- **520 Pipeline Overview**: See `520_PIPELINE_COX_OVERVIEW.md` for full pipeline context
- **Join OER to Snapshots**: See `JOIN_OER_TO_SNAPSHOTS_OVERVIEW.md` for input data preparation
- **512 OER Integration**: See `512_OER_INTEGRATION_OVERVIEW.md` for OER data preparation
- **CELL 6 Flow Guide**: See `CELL6_FLOW_AND_OPTIMIZATION_GUIDE.md` for CELL 6A/6B/6C flow

Version History

- **2026-01-08**: Updated to use @ray.remote parallelization (501-style) for all groupby().apply() operations
  - Module 1: Backward lookup parallelized
  - Module 2: Grade metrics lookup parallelized
  - Module 3: Last4 metrics parallelized
  - Module 4: Ratings given lookup parallelized
  - Module 5: Already had @ray.remote, optimized to submit-all-get-all pattern
  - **Result**: 2-4x speedup, full core utilization (16 cores at ~98%)
- **2026-01-08 (Evening)**: Enhanced user experience and code quality
  - **Progress Reporting**: Added real-time progress updates to all parallel operations
    - Reports completion percentage and timestamp during result collection
    - Uses non-blocking `ray.wait()` pattern for efficient progress tracking
    - Update frequency: every 1% or 1000 tasks (whichever is more frequent)
  - **Warning Suppression**: Suppressed Ray's informational warnings about dictionary distribution
    - Warnings added to all worker functions (`*_standalone` functions)
    - Reduces log clutter while maintaining functionality

        - Warnings are harmless but can fill logs with 100,000+ messages
    - **Code Quality**: Fixed indentation errors and preserved comments throughout the codebase
- **2026-01-22**: Added TB assignment modes (forward/backward), overlap-based pool assignment, and NaT handling policy
- **2026-01-22**: Added separate self vs pool TB modes, optional precompute of both modes, and split CELL 6C/6D for pre-pool vs pool metrics

---

Summary

The `add_cum_oer_metrics_mod_working.py` system is a comprehensive, modular solution for computing cumulative OER performance metrics. With the recent update to explicit `@ray.remote` parallelization, the system now fully utilizes all available CPU cores, resulting in significant performance improvements (2-4x speedup). Real-time progress reporting provides visibility into long-running operations, and automatic warning suppression keeps logs clean and readable. The three-cell design in the 520 pipeline allows for resumable execution and selective computation, making it practical for large-scale analysis.